

REACT ROADMAP

1. Introduction to React

JSX (JavaScript XML)
Components (Functional vs Class)
Props and State
Handling Events
Conditional Rendering
Lists and Keys

2. React Hooks

useState
useEffect
useContext
useRef
useReducer
useMemo & useCallback
Custom Hooks

3. React Router

React Router Basics
Dynamic Routes & Route Params
Nested Routes
Redirects & Navigation
Protected Routes

4. State Management

Context API
Redux (Actions, Reducers, Store, Middleware)
Redux Toolkit
Zustand, Recoil, Jotai

5. Forms & Input Handling

Controlled & Uncontrolled Components
Form Validation (Formik, Yup)

6. API Handling

Fetch API & Axios
Async/Await & Promises
React Query

7. Performance Optimization

Memoization (React.memo, useMemo, useCallback)
Virtual DOM & Reconciliation
Code Splitting & Lazy Loading

8. UI Frameworks & Styling

CSS Modules
Styled Components
Tailwind CSS
Material-UI, Ant Design

9. Advanced React

Server-Side Rendering (SSR) with [Next.js](#)
Static Site Generation (SSG)
Progressive Web Apps (PWAs)
WebSockets & Real-time Data

10. Testing in React

Jest & React Testing Library
Cypress (End-to-End Testing)

11. Deployment & Hosting

Vercel, Netlify, Firebase Hosting
Dockerizing a React App

1. Introduction to React	5
 What is React?	5
 JSX (JavaScript XML)	5
 Components (Functional vs Class)	5
 Props and State	6
 Handling Events	6
 Conditional Rendering	6
 Lists and Keys	7
 Your First Exercise	7
2. React Hooks	8
 useState	8
 useEffect	9
 useContext	10
 useRef	11
 useReducer	12
 useMemo & useCallback	13
 Custom Hooks	13
 Build a Custom Hook: useTimer	14
 Goal:	14
 File Structure	14
 Step-by-Step Code	15
 Explanation	16
 Your Task:	16
3. React Router	17
 3.1 React Router Basics	17
 3.2 Dynamic Routes & Route Params	18
 3.3 Nested Routes	19
 3.4 Redirects & Navigation	20
 3.5 Protected Routes	20
4. State Management	21
 4.1 Context API (Built-in, Light-weight)	21
 Create Context	21
 Use Context	22
 Wrap your App	22
 4.2 Redux (Popular for complex state)	23
 Redux Structure	23
 Step-by-Step Setup (with Redux Toolkit)	23
 4.3 Redux Toolkit (RTK)	27
 4.4 Alternatives to Redux	27
 Zustand (Minimal, fast, readable)	27
 Recoil (Experimental from Facebook)	28
 Jotai (Atomic and minimal)	28
 Summary Table	29
5. Forms & Input Handling	30

✓ 5.1 Controlled vs Uncontrolled Components	30
🎮 Controlled Component	30
📁 Uncontrolled Component	31
✓ 5.2 Form Validation (Formik + Yup)	31
📦 Install Formik and Yup	32
✓ Example: Signup Form with Validation	32
🔄 Comparison Table	33
✓ Mini Exercise:	33
🔑 React Login Form with Formik + Yup	34
🧱 Step 1: Install Required Packages	34
📄 Step 2: Full Example Code	34
📄 Step 3: Use in App	35
✓ What You Learned	35
6. API Handling in React	36
🔄 6.1 Fetch API (Vanilla JavaScript)	36
⚙️ 6.2 Axios (Promise-based HTTP Client)	37
🚀 6.3 React Query (aka TanStack Query)	38
✓ Comparison Table	39
🔧 Practice Task	40
👥 Build a User List with Details (React Query)	40
📦 Step 1: Setup Project (if not done already)	40
🧱 Step 2: Setup React Query Provider	40
👥 Step 3: Create UserList.js	41
✓ What You Learned	42
7. Performance Optimization in React	43
🧠 7.1 Memoization Techniques	43
✓ React.memo (Component Level)	43
✓ useMemo (Value Level)	43
🧬 7.2 Virtual DOM & Reconciliation	44
✓ Lazy load components	45
🧱 7.4 React DevTools + Chrome Profiler	45
✓ Summary of Optimization Tips	46
🔧 Mini Practice	46
8. UI Frameworks & Styling in React	46
🎯 8.1 CSS Modules	47
📁 Example	47
🔧 8.2 Styled Components (CSS-in-JS)	48
Example:	48
🔗 8.3 Tailwind CSS (Utility-First CSS)	49
Example:	49
🧱 8.4 Component Libraries	50
✓ Material UI (MUI)	50
Example:	50
✓ Ant Design	51

Example:	51
 Summary Table	51
 Mini Task:	52
9. Advanced React Concepts	52
 9.1 Server-Side Rendering (SSR) with Next.js	52
 Setup	53
 Example Page (SSR)	53
 9.2 Static Site Generation (SSG)	53
 9.3 Progressive Web Apps (PWA)	54
 Setup PWA in CRA:	54
 9.4 WebSockets & Real-Time Data	55
 Install a library like socket.io:	55
Example (Client-Side):	55
 Summary Table	57
10. Testing in React	57
 10.1 Jest + React Testing Library	58
 Setup (CRA already includes it)	58
 Example Component	58
 Test File	58
 Test DOM Expectations	59
 10.2 Cypress (End-to-End Testing)	59
 Install Cypress:	59
 Example Test	59
 Summary: When to Use What?	60
 Mini Challenge:	60
11. Deployment & Hosting in React	61
(with Vercel, AWS, and Docker)	61
 11.1 Vercel – Zero-config Hosting for React/Next.js	61
 Steps to Deploy a React App	61
 Features	61
 11.2 AWS – Flexible & Scalable Hosting	62
 Option A: AWS Amplify	62
 Option B: Amazon S3 + CloudFront (for static sites)	62
 11.3 Docker – Containerizing React for Production	63
 Why Docker?	63
 Step-by-Step: Dockerizing a React App	63
 Deploy Docker Container to:	64
 Summary Comparison	64

1. Introduction to React

What is React?

React is a JavaScript library for building **user interfaces (UIs)**, especially **single-page applications (SPAs)** where you dynamically update content without reloading the whole page. It was created by **Facebook** and follows a component-based architecture.

JSX (JavaScript XML)

JSX is a syntax extension that allows writing HTML-like code inside JavaScript.

```
const element = <h1>Hello, world!</h1>;
```

- JSX gets compiled to `React.createElement(...)`.
- It allows you to embed expressions using `{}`:

```
const name = "Karthick";  
const greeting = <h1>Hello, {name}!</h1>;
```

Components (Functional vs Class)

Functional Component (Modern Way)

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

Class Component (Old Way)

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}</h1>;  
  }  
}
```

Props and State

Props (Read-only data passed from parent to child):

```
function Greet(props) {  
  return <p>Hi, {props.name}</p>;  
}
```

State (Internal data that can change over time):

```
import { useState } from 'react';
```

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <>  
      <p>Count: {count}</p>  
      <button onClick={() => setCount(count + 1)}>Increment</button>  
    </>  
  );  
}
```

Handling Events

```
function ClickButton() {  
  function handleClick() {  
    alert('Button clicked!');  
  }  
  
  return <button onClick={handleClick}>Click me</button>;  
}
```

Conditional Rendering

```
function Greeting({ isLoggedIn }) {  
  return isLoggedIn ? <h1>Welcome back!</h1> : <h1>Please sign in.</h1>;  
}
```

Lists and Keys

```
function TodoList() {
  const todos = ['Buy milk', 'Go gym', 'Read React'];

  return (
    <ul>
      {todos.map((item, index) => (
        <li key={index}>{item}</li> // key helps React identify items
      ))}
    </ul>
  );
}
```

Your First Exercise

Create a file `App.js` and paste this:

```
import React, { useState } from 'react';

function App() {
  const [name, setName] = useState('React Learner');

  return (
    <div>
      <h1>Hello, {name}</h1>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Enter your name"
      />
    </div>
  );
}

export default App;
```

Then run:

```
npx create-react-app react-intro
cd react-intro
npm start
```

2. React Hooks

Hooks are special functions in React that let you “hook into” React features from **functional components**. They were introduced in React 16.8.

useState

`useState` lets a component **"remember" information** (i.e., maintain state).

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0); // [value, updater function]

  return (
    <div>
      <p>You clicked {count} times.</p>
      <button onClick={() => setCount(count + 1)}>Click</button>
    </div>
  );
}
```

 Concept:

- `useState(0)` initializes `count` to 0
 - `setCount` updates `count`
 - Changing state causes **re-render**
-

useEffect

useEffect is used for **side effects** (like API calls, subscriptions, timers).

```
import { useState, useEffect } from 'react';

function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => setSeconds(s => s + 1), 1000);

    return () => clearInterval(interval); // cleanup on unmount
  }, []);

  return <p>Timer: {seconds}s</p>;
}
```

Concept:

- Runs **after** render
 - Empty `[]` runs only once (on mount)
 - You can clean up effects (e.g., clear timer)
-

useContext

`useContext` allows components to **access global values** like a theme, user, etc.

```
const ThemeContext = React.createContext('light');

function ThemedButton() {
  const theme = React.useContext(ThemeContext);
  return <button className={theme}>I am {theme} themed</button>;
}

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <ThemedButton />
    </ThemeContext.Provider>
  );
}
```

 Concept:

- Like a mini global store for deeply nested components.
-

useRef

`useRef` gives you a **mutable reference** that persists across renders. Often used to:

- Reference a DOM element
- Store a value without triggering re-renders

```
import { useRef } from 'react';

function FocusInput() {
  const inputRef = useRef(null);

  const focus = () => inputRef.current.focus();

  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={focus}>Focus Input</button>
    </div>
  );
}
```

useReducer

`useReducer` is an alternative to `useState` when state logic becomes complex.

```
import { useReducer } from 'react';

function reducer(state, action) {
  switch (action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'decrement': return { count: state.count - 1 };
    default: return state;
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });

  return (
    <>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
    </>
  );
}
```

 Good for managing complex state like forms, modals, etc.

useMemo & useCallback

useMemo: Caches the result of expensive computations

```
const expensiveValue = useMemo(() => slowFunction(input), [input]);
```

useCallback: Caches a function reference

```
const memoizedCallback = useCallback(() => {  
  doSomething(input);  
}, [input]);
```

These help avoid **unnecessary recalculations or re-renders**, especially with child components.

Custom Hooks

You can create your own reusable hooks.

```
function useCounter(initial = 0) {  
  const [count, setCount] = useState(initial);  
  const increment = () => setCount(count + 1);  
  const decrement = () => setCount(count - 1);  
  return { count, increment, decrement };  
}  
  
function App() {  
  const { count, increment, decrement } = useCounter(10);  
  
  return (  
    <>  
      <h1>{count}</h1>  
      <button onClick={increment}>+</button>  
      <button onClick={decrement}>-</button>  
    </>  
  );  
}
```

✅ **Mini Project Idea:** Create a `useTimer` custom hook that tracks time passed.

Awesome! Let's now build a simple **custom hook** called `useTimer` step by step.

Build a Custom Hook: `useTimer`

Goal:

We'll create a hook that:

- Starts a timer
- Increases every second
- Lets you start, pause, and reset the timer

File Structure

```
src/  
├── hooks/  
│   └── useTimer.js  
└── App.js
```

Step-by-Step Code

 **src/hooks/useTimer.js**

```
import { useEffect, useRef, useState } from 'react';

function useTimer() {
  const [seconds, setSeconds] = useState(0);
  const [isRunning, setIsRunning] = useState(false);
  const intervalRef = useRef(null);

  useEffect(() => {
    if (isRunning) {
      intervalRef.current = setInterval(() => {
        setSeconds(prev => prev + 1);
      }, 1000);
    }

    return () => clearInterval(intervalRef.current);
  }, [isRunning]);

  const start = () => setIsRunning(true);
  const pause = () => setIsRunning(false);
  const reset = () => {
    setIsRunning(false);
    setSeconds(0);
  };

  return { seconds, isRunning, start, pause, reset };
}

export default useTimer;
```

src/App.js

```
import React from 'react';
import useTimer from './hooks/useTimer';

function App() {
  const { seconds, isRunning, start, pause, reset } = useTimer();

  return (
    <div style={{ textAlign: 'center', marginTop: '2rem' }}>
      <h1>⌚ Timer: {seconds}s</h1>
      <button onClick={start} disabled={!isRunning}>Start</button>
      <button onClick={pause} disabled={isRunning}>Pause</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
}

export default App;
```

Explanation

- `useRef` holds the interval ID — we avoid re-setting on every render.
- `useEffect` starts or stops the interval based on `isRunning`.
- We return everything needed from `useTimer` for the component to control.

Your Task:

Try enhancing this:

- Add lap functionality (record times in an array)
 - Format `seconds` into `hh:mm:ss`
 - Add auto-start or countdown feature
-

3. React Router

React Router is a **library** used to create navigation between different "pages" in a React app—without reloading the browser (single-page app behavior).

 Install React Router:

```
npm install react-router-dom
```

3.1 React Router Basics

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
```

```
function Home() {  
  return <h2> Home Page</h2>;  
}
```

```
function About() {  
  return <h2> About Page</h2>;  
}
```

```
function App() {  
  return (  
    <BrowserRouter>  
      <nav>  
        <Link to="/">Home</Link> | { ' '  
        <Link to="/about">About</Link>  
      </nav>  
  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/about" element={<About />} />  
      </Routes>  
    )  
}
```

```
    </BrowserRouter>
  );
}
```

Explanation:

- **BrowserRouter**: Wraps your app and enables routing.
- **Routes**: Contains multiple **Routes**.
- **Route**: Maps a **path** to a component.
- **Link**: Navigates without refreshing the page.

3.2 Dynamic Routes & Route Params

You can pass values (like IDs) in the URL.

```
function UserProfile({ id }) {
  return <h3>Viewing user ID: {id}</h3>;
}
```

```
// Accessing it from the route
import { useParams } from 'react-router-dom';
```

```
function UserPage() {
  const { userId } = useParams(); // e.g., /users/7 → { userId: "7" }
  return <h2>User ID: {userId}</h2>;
}
```

```
// App.js
<Route path="/users/:userId" element={<UserPage />} />
```

🧩 3.3 Nested Routes

Let's say a **Dashboard** has inner pages like **Stats** and **Settings**.

```
function Dashboard() {  
  return (  
    <>  
    <h1>Dashboard</h1>  
    <nav>  
      <Link to="stats">Stats</Link> | <Link to="settings">Settings</Link>  
    </nav>  
    <Outlet />  
  </>  
);  
}  
  
function Stats() {  
  return <p>📊 Statistics here</p>;  
}  
  
function Settings() {  
  return <p>⚙️ Settings here</p>;  
}  
  
<Routes>  
  <Route path="/dashboard" element={<Dashboard />}>  
    <Route path="stats" element={<Stats />} />  
    <Route path="settings" element={<Settings />} />  
  </Route>  
</Routes>
```

🔑 **Outlet** is where child routes render inside the parent.

3.4 Redirects & Navigation

You can programmatically navigate using `useNavigate`.

```
import { useNavigate } from 'react-router-dom';

function Login() {
  const navigate = useNavigate();

  const handleLogin = () => {
    // perform login logic
    navigate('/dashboard');
  };

  return <button onClick={handleLogin}>Login</button>;
}
```

3.5 Protected Routes

You can protect a route so only logged-in users can access it.

```
function ProtectedRoute({ isLoggedIn, children }) {
  return isLoggedIn ? children : <Navigate to="/login" />;
}
```

// Usage

```
<Routes>
  <Route path="/dashboard" element={
    <ProtectedRoute isLoggedIn={true}>
      <Dashboard />
    </ProtectedRoute>
  } />
</Routes>
```

```
} />
</Routes>
```

✓ Mini Practice: Navigation App

Try building this:

- `/` → Home Page
- `/about` → About Page
- `/profile/:name` → User Profile
- `/dashboard/stats` and `/dashboard/settings`
- Use a navbar with `Link`

4. State Management

As your app grows, managing **shared state** (state across multiple components) becomes complex. That's where **state management libraries** come in.

4.1 Context API (Built-in, Light-weight)

Great for **global values** like themes, user login, language, etc.

Create Context

```
import React, { createContext, useContext, useState } from 'react';

const UserContext = createContext();

export function UserProvider({ children }) {

  const [user, setUser] = useState({ name: "Karthick", loggedIn:
true });

  return (

    <UserContext.Provider value={{ user, setUser }}>

      {children}

    </UserContext.Provider>
```

```
);  
}
```

Use Context

```
import { useContext } from 'react';  
import { UserContext } from '../UserContext';  
function Profile() {  
  const { user } = useContext(UserContext);  
  return <h2>Hello, {user.name}!</h2>;  
}
```

Wrap your App

```
import { UserProvider } from '../UserContext';  
  
function App() {  
  return (  
    <UserProvider>  
      <Profile />  
    </UserProvider>  
  );  
}
```

}

4.2 Redux (Popular for complex state)

Redux is a powerful state container.

 Install Redux & React bindings:

```
npm install @reduxjs/toolkit react-redux
```

Redux Structure

- **Actions** → describe what happened
 - **Reducers** → describe how state changes
 - **Store** → keeps all app state
 - **Dispatch** → sends actions to reducers
-

Step-by-Step Setup (with Redux Toolkit)

1. Create a slice

```
// store/userSlice.js
```

```
import { createSlice } from '@reduxjs/toolkit';
```

```

const userSlice = createSlice({
  name: 'user',
  initialState: { name: 'Guest', loggedIn: false },
  reducers: {
    login(state, action) {
      state.name = action.payload;
      state.loggedIn = true;
    },
    logout(state) {
      state.name = 'Guest';
      state.loggedIn = false;
    }
  }
});

```

```

export const { login, logout } = userSlice.actions;
export default userSlice.reducer;

```

2. Create the Redux store

```

// store/index.js

import { configureStore } from '@reduxjs/toolkit';
import userReducer from './userSlice';

const store = configureStore({
  reducer: { user: userReducer }
});

```



```
export default store;
```

3. Provide the store to your app

```
import { Provider } from 'react-redux';  
import store from './store';  
function App() {  
  return (  
    <Provider store={store}>  
      <MyComponent />  
    </Provider>  
  );  
}
```

4. Use in a component

```
import { useSelector, useDispatch } from 'react-redux';

import { login, logout } from './store/userSlice';

function MyComponent() {

  const user = useSelector((state) => state.user);

  const dispatch = useDispatch();

  return (

    <div>

      <h2>Welcome, {user.name}</h2>

      {user.loggedIn ? (

        <button onClick={() => dispatch(logout())}>Logout</button>

      ) : (

        <button onClick={() =>
dispatch(login('Karthick'))}>Login</button>

      )}

    </div>

  );
```

```
}
```

4.3 Redux Toolkit (RTK)

We already used RTK above! It's:

- Simpler syntax
- Comes with `createSlice`, `createAsyncThunk`
- Handles immutability internally (via Immer)

You **don't need** classic `combineReducers`, `createStore`, etc.

4.4 Alternatives to Redux

Sometimes Redux is too heavy for small to medium apps. Consider:

Zustand (Minimal, fast, readable)

```
npm install zustand
```

```
import create from 'zustand';
```

```
const useStore = create((set) => ({
```

```
    count: 0,

    increment: () => set((state) => ({ count: state.count + 1 })))
  }));

function Counter() {
  const { count, increment } = useStore();

  return (
    <>
      <h1>{count}</h1>
      <button onClick={increment}>+</button>
    </>
  );
}
```

Recoil (Experimental from Facebook)

Recoil allows shared and derived state across components using **atoms** and **selectors**.

```
npm install recoil
```

Jotai (Atomic and minimal)

Like Recoil, but simpler. Each state value is an “atom.”

Summary Table

Feature	Context API	Redux Toolkit	Zustand	Recoil
Boilerplate	Low	Medium	Very Low	Low
Async Support	Manual	Built-in	Manual	Built-in
Performance	Decent	High	High	High
Learning Curve	Easy	Moderate	Easy	Moderate

5. Forms & Input Handling

Forms are a **core** part of any web app (login, registration, search, etc.). In React, form handling is done using two approaches:

5.1 Controlled vs Uncontrolled Components

Controlled Component

- React manages the form state using `useState`.

```
import { useState } from 'react';

function ControlledForm() {
  const [name, setName] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    alert(`Hello, ${name}!`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Enter name"
      />
    </form>
  );
}
```

```

    <button type="submit">Submit</button>
  </form>
);
}

```

🧠 Each change in input updates the state → rerenders the component.

📦 Uncontrolled Component

- You use `useRef` to access the DOM element directly.

```

import { useRef } from 'react';

function UncontrolledForm() {
  const inputRef = useRef();

  const handleSubmit = (e) => {
    e.preventDefault();
    alert(`You typed: ${inputRef.current.value}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input ref={inputRef} placeholder="Enter something" />
      <button type="submit">Submit</button>
    </form>
  );
}

```

🧠 Good when you need raw DOM control, but not common in modern apps.

✅ 5.2 Form Validation (Formik + Yup)

React alone doesn't handle **validation** well for complex forms.

📦 Install Formik and Yup

npm install formik yup

✓ Example: Signup Form with Validation

```
import { Formik, Form, Field, ErrorMessage } from 'formik';
import * as Yup from 'yup';

const SignupSchema = Yup.object({
  name: Yup.string().required('Name is required'),
  email: Yup.string().email('Invalid email').required('Email required'),
  password: Yup.string().min(6, 'Minimum 6 characters').required(),
});

function SignupForm() {
  return (
    <Formik
      initialValues={{ name: '', email: '', password: '' }}
      validationSchema={SignupSchema}
      onSubmit={values => alert(JSON.stringify(values, null, 2))}
    >
      <Form>
        <div>
          <label>Name:</label>
          <Field name="name" />
          <ErrorMessage name="name" component="div" style={{ color: 'red' }} />
        </div>

        <div>
          <label>Email:</label>
          <Field name="email" />
          <ErrorMessage name="email" component="div" style={{ color: 'red' }} />
        </div>

        <div>
          <label>Password:</label>
```



```

    <Field name="password" type="password" />
    <ErrorMessage name="password" component="div" style={{ color: 'red' }} />
  </div>

  <button type="submit">Register</button>
</Form>
</Formik>
);
}

```

Comparison Table

Feature	Controlled	Uncontrolled	Formik + Yup
State Handling	useState	useRef	Internal by Formik
Validation	Manual	Manual	Easy with Yup
Use Case	Small forms	Simple DOM	Scalable forms

Mini Exercise:

Build a login form using:

- Controlled inputs
 - Email & password validation using Yup
 - Submit with alert showing the user input
-

Great! Let's build a **login form** step by step using:

-  Controlled Components
-  Validation with **Formik + Yup**
-  Alert on form submit



React Login Form with Formik + Yup



Step 1: Install Required Packages

npm install formik yup



Step 2: Full Example Code

```
// LoginForm.js
import React from 'react';
import { Formik, Form, Field, ErrorMessage } from 'formik';
import * as Yup from 'yup';

// 🎯 Validation Schema
const LoginSchema = Yup.object().shape({
  email: Yup.string()
    .email('Invalid email format')
    .required('Email is required'),
  password: Yup.string()
    .min(6, 'Password must be at least 6 characters')
    .required('Password is required'),
});

function LoginForm() {
  return (
    <div style={{ maxWidth: '400px', margin: '2rem auto', padding: '1rem', border: '1px solid #ccc' }}>
      <h2>Login</h2>

      <Formik
        initialValues={{ email: '', password: '' }}
        validationSchema={LoginSchema}
        onSubmit={values => {
          alert(`Email: ${values.email}\nPassword: ${values.password}`);
        }}
      >
        {{{ isSubmitting }}} => (
          <Form>
```

```

    <div>
      <label>Email</label>
      <Field type="email" name="email" />
      <ErrorMessage name="email" component="div" style={{ color: 'red' }} />
    </div>

    <div>
      <label>Password</label>
      <Field type="password" name="password" />
      <ErrorMessage name="password" component="div" style={{ color: 'red' }} />
    </div>

    <button type="submit" disabled={isSubmitting}>Login</button>
  </Form>
)}
</Formik>
</div>
);
}

export default LoginForm;

```

Step 3: Use in App

```

import React from 'react';
import LoginForm from './LoginForm';

function App() {
  return (
    <div>
      <LoginForm />
    </div>
  );
}

export default App;

```

What You Learned

- Controlled form inputs using Formik's **Field**
- Schema-based validation using Yup

- Inline error display with `ErrorMessage`
 - Easy submit handler (`onSubmit`)
-

6. API Handling in React

Fetching data from a backend is essential in real-world apps — login, display data, update, delete, etc.

We'll cover:

1.  Fetch API (built-in)
 2.  Axios (popular library)
 3.  React Query (powerful data-fetching library)
-



6.1 Fetch API (Vanilla JavaScript)

```
import { useEffect, useState } from 'react';
```

```
function Users() {  
  const [users, setUsers] = useState([]);  
  
  useEffect(() => {  
    fetch('https://jsonplaceholder.typicode.com/users') // dummy API  
      .then((res) => res.json())  
      .then((data) => setUsers(data));  
  }, []);  
  
  return (  
    <>  
      <h2>Users</h2>  
      <ul>  
        {users.map(user => <li key={user.id}>{user.name}</li>)}  
      </ul>  
    </>  
  );  
}
```

🧠 Good for simple cases, but not great for error handling or complex logic.

⚙️ 6.2 Axios (Promise-based HTTP Client)

Install it:

```
npm install axios
```

```
import axios from 'axios';  
import { useEffect, useState } from 'react';
```

```
function UsersAxios() {  
  const [users, setUsers] = useState([]);  
  
  useEffect(() => {  
    axios.get('https://jsonplaceholder.typicode.com/users')  
      .then((res) => setUsers(res.data))  
      .catch((err) => console.error(err));  
  }, []);  
  
  return (  
    <>  
      <h2>Users (Axios)</h2>  
      <ul>  
        {users.map(user => <li key={user.id}>{user.name}</li>)}  
      </ul>  
    </>  
  );  
}
```

✅ Benefits:

- Built-in JSON parsing
- Interceptors for auth tokens
- Better error handling

6.3 React Query (aka TanStack Query)

React Query makes **data fetching**, **caching**, **loading states**, and **background refetching** super easy.

Install it:

```
npm install @tanstack/react-query
```

Setup provider in `App.js`:

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import Users from './Users';
```

```
const queryClient = new QueryClient();
```

```
function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <Users />
    </QueryClientProvider>
  );
}
```

Now create `Users.js`:

```
import { useQuery } from '@tanstack/react-query';
import axios from 'axios';
```

```
function Users() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['users'],
    queryFn: () =>
      axios.get('https://jsonplaceholder.typicode.com/users')
        .then(res => res.data)
  });
}
```

```

});

if (isLoading) return <p>Loading...</p>;
if (error) return <p>Error: {error.message}</p>;

return (
  <>
    <h2>Users (React Query)</h2>
    <ul>
      {data.map(user => <li key={user.id}>{user.name}</li>)}
    </ul>
  </>
);
}

```

🔥 Features:

- Automatic caching
- Background refetching
- Loading & error state built-in
- Retry, pagination, mutations

✅ Comparison Table

Feature	Fetch	Axios	React Query
Easy to use	✅	✅✅	✅✅✅ (after setup)
Caching	❌	❌	✅✅✅
Retry/Refetch	❌	❌	✅✅✅
Built-in JSON	❌	✅	✅ (via Axios or fetch)

🔧 Practice Task

Create a user list component:

- Fetch data from <https://jsonplaceholder.typicode.com/users>
 - Show loading and error state
 - Let user click a name and show their email and phone
-



Build a User List with Details (React Query)

We'll:

- Fetch users
 - Show loading/error states
 - Let user click a name to show details (email, phone)
-



Step 1: Setup Project (if not done already)

npm install @tanstack/react-query axios



Step 2: Setup React Query Provider

In `App.js`:

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import UserList from './UserList';
```

```
const queryClient = new QueryClient();
```

```
function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <UserList />
    </QueryClientProvider>
  );
}
```



```
export default App;
```

Step 3: Create **UserList.js**

```
import { useQuery } from '@tanstack/react-query';
import axios from 'axios';
import { useState } from 'react';

function UserList() {
  const [selectedUser, setSelectedUser] = useState(null);

  const { data: users, isLoading, isError, error } = useQuery({
    queryKey: ['users'],
    queryFn: () =>
      axios.get('https://jsonplaceholder.typicode.com/users')
        .then(res => res.data)
  });

  if (isLoading) return <p>Loading users...</p>;
  if (isError) return <p>Error: {error.message}</p>;

  return (
    <div style={{ display: 'flex', gap: '2rem', padding: '1rem' }}>
      <div>
        <h3>User List</h3>
        <ul>
          {users.map(user => (
            <li
              key={user.id}
              onClick={() => setSelectedUser(user)}
              style={{ cursor: 'pointer', marginBottom: '0.5rem' }}
            >
              {user.name}
            </li>
          ))}
        </ul>
      </div>

      <div>
        <h3>User Details</h3>

```

```

    {selectedUser ? (
      <div>
        <p><strong>Name:</strong> {selectedUser.name}</p>
        <p><strong>Email:</strong> {selectedUser.email}</p>
        <p><strong>Phone:</strong> {selectedUser.phone}</p>
        <p><strong>Company:</strong> {selectedUser.company.name}</p>
      </div>
    ) : (
      <p>Select a user to see details</p>
    )}
  </div>
</div>
);
}

```

export default UserList;

✓ What You Learned

- Fetch users from an API using React Query
- Show loading and error state
- Handle click and show detailed info
- Use `useState` for local selection state

🎯 Want a challenge?

Try:

- Adding a search bar to filter users
 - Fetching detailed user info from `/users/:id`
 - Adding pagination with `React Query`
-

7. Performance Optimization in React

As your app grows, unnecessary re-renders or expensive computations can slow it down. Here are key techniques to keep things fast:

7.1 Memoization Techniques

React.memo (Component Level)

Prevents re-rendering of a component if props don't change.

```
const Greeting = React.memo(function Greeting({ name }) {  
  console.log("Greeting re-rendered");  
  return <h2>Hello, {name}!</h2>;  
});
```

Use `React.memo()` when:

- Component renders same output for same props
 - Re-renders hurt performance
-

useMemo (Value Level)

Memoizes a **calculated value**.

```
import { useMemo, useState } from 'react';
```

```
function ExpensiveCalculation({ num }) {  
  const result = useMemo(() => {  
    console.log("Calculating...");  
    let total = 0;  
    for (let i = 0; i < 100000000; i++) {  
      total += num;  
    }  
  });  
  return result;  
}
```

```

    }
    return total;
  }, [num]);

  return <p>Result: {result}</p>;
}

```

🧠 `useMemo` only recalculates when `num` changes.

✅ `useCallback` (Function Memoization)

Memoizes a **function** so it doesn't change across renders.

```

import { useState, useCallback } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  const increment = useCallback(() => setCount(c => c + 1), []);

  return (
    <>
      <h2>{count}</h2>
      <button onClick={increment}>+</button>
    </>
  );
}

```

✅ `useCallback` is helpful when:

- Passing functions to child components
- Preventing unnecessary re-renders

7.2 Virtual DOM & Reconciliation

React uses a **Virtual DOM**:

- When state/props change, React creates a new virtual tree
- Compares it (diffing) with the old one
- Efficiently updates only what's changed

🧠 You don't control this directly, but you optimize by:

- Using keys in lists properly
- Avoiding anonymous functions or unnecessary inline objects
- Splitting components

7.3 Code Splitting & Lazy Loading

Split code into smaller chunks so users load only what they need.

Lazy load components

```
import { Suspense, lazy } from 'react';

const LazyComponent = lazy(() => import('./LazyComponent'));

function App() {
  return (
    <Suspense fallback={<p>Loading...</p>}>
      <LazyComponent />
    </Suspense>
  );
}
```

Use this to:

- Split routes (e.g., lazy load [/dashboard](#))
- Delay loading heavy components

7.4 React DevTools + Chrome Profiler

Use **React DevTools** to:

- Visualize re-renders
- Highlight unnecessary updates
- Profile component performance

 Install as a Chrome extension → "React Developer Tools"

✓ Summary of Optimization Tips

Technique	Use When
<code>React.memo</code>	Child re-renders unnecessarily
<code>useMemo</code>	Expensive calculation needs caching
<code>useCallback</code>	Function passed to memoized child component
Lazy Loading	Routes, components, or libraries not used upfront
Virtual DOM Keys	Rendering lists or dynamic components

Mini Practice

- Wrap a component with `React.memo`
- Use `useMemo` to cache a Fibonacci or factorial calculator
- Lazy load a separate route/page (e.g., `About.js`)

Styling is what gives your app its look and feel, and React supports many ways to style your UI.

8. UI Frameworks & Styling in React

We'll explore:

1. ✓ CSS Modules
2. ✓ Styled Components

3.  Tailwind CSS
 4.  Component Libraries (Material-UI, Ant Design)
-

8.1 CSS Modules

Scoped styles — styles are only applied to the component they belong to.

Example

Button.module.css

```
.btn {  
  
  background-color: royalblue;  
  
  color: white;  
  
  padding: 10px 20px;  
  
  border: none;  
  
  border-radius: 8px;  
  
}
```

Button.js

```
import styles from './Button.module.css';  
  
function Button() {  
  
  return <button className={styles.btn}>Click Me</button>;  
  
}
```

 Pros:

- No global CSS collisions
 - Supported out of the box with Create React App
-

8.2 Styled Components (CSS-in-JS)

Style your components with actual JavaScript!

 Install:

```
npm install styled-components
```

Example:

```
import styled from 'styled-components';
```

```
const FancyButton = styled.button`
```

```
  background-color: hotpink;
```

```
  color: white;
```

```
  padding: 10px 20px;
```

```
  border-radius: 10px;
```

```
  font-size: 18px;
```

```
`;
```

```
function App() {
```

```
  return <FancyButton>Styled Button</FancyButton>;
```

```
}
```


✓ Pros:

- Dynamically adjust styles using props
 - No class name conflicts
-

🔧 8.3 Tailwind CSS (Utility-First CSS)

Write classes directly in JSX — no CSS files needed.

📦 Setup in Create React App:

```
npm install -D tailwindcss postcss autoprefixer
```

```
npx tailwindcss init
```

Then update:

- `tailwind.config.js`
- `index.css` with Tailwind's base, components, and utilities

Example:

```
function TailwindButton() {  
  return (  
    <button className="bg-indigo-600 text-white px-4 py-2 rounded-lg">  
      Tailwind Button  
    </button>  
  );  
}
```

✓ Pros:

- Fast UI prototyping
 - Clean, utility-based structure
 - Mobile-first design ready
-

8.4 Component Libraries

Material UI (MUI)

 Install:

```
npm install @mui/material @emotion/react @emotion/styled
```

Example:

```
import Button from '@mui/material/Button';
```

```
function MUIButton() {  
  return <Button variant="contained" color="primary">MUI Button</Button>;  
}
```

 Comes with:

- Ready-to-use components (Cards, Modals, Tables)
 - Theming support
 - Responsiveness
-

Ant Design

 Install:

```
npm install antd
```

Example:

```
import { Button } from 'antd';

function AntButton() {
  return <Button type="primary">Ant Design Button</Button>;
}
```

 Includes:

- Grid systems
- Forms
- Date pickers, charts, etc.

Summary Table

Styling Method	Scoped	Dynamic Styles	Theming	Ready-made UI
CSS Modules				
Styled Components				
Tailwind CSS		Utility-based		
Material-UI / AntD				  

Mini Task:

Try creating a small UI:

- A card component
- Use Tailwind to style it
- Add a Material UI button inside it

9. Advanced React Concepts

These topics elevate your app to a professional production level — helping with SEO, performance, offline support, and real-time updates.

We'll cover:

1.  Server-Side Rendering (SSR)
2.  Static Site Generation (SSG)
3.  Progressive Web Apps (PWA)
4.  WebSockets & Real-Time Data

9.1 Server-Side Rendering (SSR) with Next.js

SSR means rendering the page on the **server** before it reaches the browser — improves SEO and faster initial load.

 Use **Next.js**, a React framework that supports SSR out-of-the-box.

Setup

```
npx create-next-app@latest my-next-app
```

```
cd my-next-app
```

```
npm run dev
```

Example Page (SSR)

```
// pages/index.js
```

```
export async function getServerSideProps() {  
  
  const res = await fetch('https://jsonplaceholder.typicode.com/posts/1');  
  
  const post = await res.json();  
  
  return { props: { post } };  
}
```

```
export default function Home({ post }) {  
  
  return (  
  
    <div>  
  
      <h1>{post.title}</h1>  
  
      <p>{post.body}</p>  
  
    </div>  
  
  );  
}
```

✓ Runs every time the page is requested.

9.2 Static Site Generation (SSG)

SSG means building the HTML **at build time** (not on every request) — best for speed and caching.

```
// pages/blog.js
```

```
export async function getStaticProps() {  
  
  const res = await fetch('https://jsonplaceholder.typicode.com/posts/1');  
  
  const post = await res.json();  
  
  return { props: { post } };  
}
```

```
}
```

```
export default function Blog({ post }) {  
  
  return (  
  
    <div>  
  
      <h1>Static: {post.title}</h1>  
  
    </div>  
  
  ); }  
}
```

9.3 Progressive Web Apps (PWA)

A PWA is a web app that:

- Works offline
- Can be installed on your phone
- Loads fast

Setup PWA in CRA:

```
npx create-react-app my-pwa --template cra-template-pwa
```

Now your app:

- Caches files with a service worker
- Works offline
- Can be installed on mobile

✓ You can customize this in `service-worker.js`.

9.4 WebSockets & Real-Time Data

Use **WebSockets** for live updates (chat, notifications, etc.)

 Install a library like **socket.io**:

```
npm install socket.io-client
```

Example (Client-Side):

```
import { useEffect, useState } from 'react';
```

```
import { io } from 'socket.io-client';
```

```
const socket = io('http://localhost:4000');
```

```
function Chat() {
```

```
  const [message, setMessage] = useState("");
```

```
  const [messages, setMessages] = useState([]);
```

```
  useEffect(() => {
```

```
    socket.on('chat message', (msg) => {
```

```
      setMessages((prev) => [...prev, msg]);
```

```
    });
```

```
    return () => socket.off('chat message');
```

```
  }, []);
```

```

const sendMessage = () => {
  socket.emit('chat message', message);
  setMessage("");
};

return (
  <div>
    <ul>{messages.map((m, i) => <li key={i}>{m}</li>)}</ul>
    <input value={message} onChange={(e) => setMessage(e.target.value)} />
    <button onClick={sendMessage}>Send</button>
  </div>
);
}

```

 On the server, you'll run something like:

```
// Node.js Express + Socket.io server
```

```
const io = require('socket.io')(server);
```

```

io.on('connection', (socket) => {
  socket.on('chat message', (msg) => {
    io.emit('chat message', msg);
  });
});

```

 Use for live updates, multiplayer games, dashboards, notifications.

Summary Table

Feature	Best For	Tool/Lib
SSR	SEO, Dynamic Pages	Next.js (<code>getServerSideProps</code>)
SSG	Static Sites, Blogs	Next.js (<code>getStaticProps</code>)
PWA	Offline access, Installable	CRA PWA template
WebSockets	Real-time chat, live updates	socket.io-client

10. Testing in React

Testing ensures your app works correctly and is **maintainable, bug-free, and production-ready**.

We'll cover:

1.  Unit & Integration Testing with **Jest + React Testing Library (RTL)**
 2.  End-to-End Testing with **Cypress**
-

10.1 Jest + React Testing Library

These help test **React components** in isolation (unit tests) or together (integration).

Setup (CRA already includes it)

If you're using Create React App, it's pre-configured.

Otherwise:

```
npm install --save-dev jest @testing-library/react @testing-library/jest-dom
```

Example Component

```
// Button.js
function Button({ label, onClick }) {
  return <button onClick={onClick}>{label}</button>;
}
export default Button;
```

Test File

```
// Button.test.js
import { render, screen, fireEvent } from '@testing-library/react';
import Button from './Button';

test('renders the button with label', () => {
  render(<Button label="Click Me" onClick={() => {}} />);
  const btn = screen.getByText(/click me/i);
  expect(btn).toBeInTheDocument();
});

test('calls onClick when clicked', () => {
  const mockFn = jest.fn();
  render(<Button label="Click" onClick={mockFn} />);
  fireEvent.click(screen.getByText('Click'));
  expect(mockFn).toHaveBeenCalledTimes(1);
});
```

Use cases:

- Render a component

- Check text/content
- Simulate events
- Test props or internal logic

✓ Test DOM Expectations

Using `@testing-library/jest-dom` you can do:

```
expect(element).toBeInTheDocument();  
expect(button).toBeDisabled();  
expect(link).toHaveAttribute('href', '/home');
```

10.2 Cypress (End-to-End Testing)

Tests your entire app in a **real browser**, like a real user.

Install Cypress:

```
npm install cypress --save-dev  
npx cypress open
```

This opens the Cypress GUI for writing E2E tests.

✓ Example Test

```
// cypress/e2e/app.cy.js  
describe('Homepage', () => {  
  it('loads and shows greeting', () => {  
    cy.visit('http://localhost:3000');  
    cy.contains('Welcome').should('be.visible');  
  });  
  it('clicks a button', () => {  
    cy.get('button').click();  
    cy.contains('Clicked!').should('exist');  
  });  
});
```

✓ Cypress can:

- Visit pages
- Interact with buttons, forms, links
- Simulate real user behavior
- Run full workflows

Summary: When to Use What?

Tool	Use For	Example
Jest	Unit Testing (JS logic, utils)	Validate login function
React Testing Library	Component Testing	Check button renders, clicks
Cypress	Full App Flow (E2E)	Fill form, login, navigate

Mini Challenge:

Try testing this **Counter** component with RTL:

```
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </>
  );
}
```

 Test for:

- Rendering the count
 - Clicking the button increases count
-

11. Deployment & Hosting in React

(with Vercel, AWS, and Docker)

11.1 Vercel – *Zero-config Hosting for React/Next.js*

Vercel is ideal for fast, serverless deployment — perfect for React or Next.js apps.

Steps to Deploy a React App

1. Push your project to **GitHub**
2. Go to <https://vercel.com>
3. Log in → Click “**Add New Project**”
4. Import your GitHub repo
5. Vercel auto-detects:
 - **Build Command:** `npm run build`
 - **Output Directory:** `build`
6. Click **Deploy**

Features

- Free tier with **CI/CD**
 - **Custom domains**
 - Auto-redeployment on every Git push
 - Optimized for Next.js but supports any frontend
-

11.2 AWS – *Flexible & Scalable Hosting*

Option A: AWS Amplify

A high-level platform from AWS — perfect for React apps.

Steps:

1. Push project to GitHub
2. Go to <https://aws.amazon.com/amplify>
3. Connect GitHub repository
4. Configure:
 - Build command: `npm run build`
 - Output folder: `build/`
5. Deploy

-  Free tier for 1000 builds/month
 -  Hosting + CI/CD + custom domain
 -  Optional backend: GraphQL, Auth, Storage
-

Option B: Amazon S3 + CloudFront (for static sites)

1. **Build your app**

`npm run build`

2. **Upload contents of `/build` to S3**
3. **Enable static website hosting** in the S3 bucket settings.
4. (Optional) **Use CloudFront** to:
 - Add a custom domain

- Enable CDN and HTTPS

 *Best for: simple static hosting with global delivery*

11.3 Docker – Containerizing React for Production

Why Docker?

- Environment-agnostic (same behavior on any server)
 - Use in CI/CD, Kubernetes, cloud platforms
-

Step-by-Step: Dockerizing a React App

1. Create a **Dockerfile**:

```
# Step 1: Build the app
FROM node:18-alpine as build
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build

# Step 2: Serve with Nginx
FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

2. Build & Run Docker Image:

```
docker build -t my-react-app .
docker run -p 3000:80 my-react-app
```

 Opens app at <http://localhost:3000>

✅ Deploy Docker Container to:

- AWS ECS or Fargate
- EC2 with Docker installed
- DigitalOcean App Platform
- Kubernetes (EKS/GKE)

📋 Summary Comparison

Feature	Vercel	AWS Amplify / S3	Docker
CI/CD	✅ Built-in	✅ Amplify only	❌ Manual or GitHub Actions
Custom Domain	✅ Free	✅ Yes	✅ With Nginx config
HTTPS/SSL	✅ Auto	✅ (manual via CloudFront)	✅ (with reverse proxy)
Use Case	Fast frontend deploy	Static sites or full-stack	Full control, DevOps
Complexity	📖 Beginner-friendly	⚙️ Moderate	🧠 Advanced
